

Dynamic Anytime Scheduling for LLM Inference: Bounding Tail Latency via Predictive Early-Exit Mechanisms

Nithin Palyam

Department of Computer Science and Engineering
University of South Florida
Tampa, FL, USA
palyam@usf.edu

Abstract—Standard autoregressive large language model (LLM) inference processes every transformer layer for every generated token, producing per-token latencies that are highly variable and potentially unbounded in the worst case under growing context. In latency-sensitive applications such as clinical decision support and cyber-physical systems, this tail latency violates hard real-time constraints. This paper presents an anytime algorithm framework for LLM token generation that exploits the predictive signal present in early transformer layers to make deadline-aware exit decisions. We implement two schedulers on TinyLlama-1.1B-Chat: a stateless two-pass dynamic threshold scheduler and a KV-cached single-pass scheduler using a forward hook to capture intermediate representations. Profiling on a bare-metal RTX 4000 Ada GPU (21 GB VRAM) demonstrates that the KV-cached scheduler achieves P99 TPOT of 25.2 ms against a hard deadline of $D = 45$ ms (utilization $U = 0.56$), with zero deadline misses across 240 token samples on chat-template formatted clinical prompts. In contrast, the stateless scheduler exceeds the deadline on 16% of tokens ($P99 = 48.3$ ms, $U = 1.07$). An exit-layer ablation study across L5 through L20 validates Layer 16 as the exit point. Extreme Value Theory (EVT) Gumbel fits establish probabilistic WCET bounds: $P(\text{TPOT} > 45 \text{ ms}) < 10^{-8}$ for sequences up to 512 tokens. Thermal profiling over 500 consecutive tokens confirms WCET stability (drift = -0.86 ms, -4.0%). Liu & Layland analysis yields $N_{\max} = 2$ concurrent schedulable inference requests under round-robin token scheduling. A 30-query PubMedQA clinical benchmark confirms 0% deadline misses at 51.2 tokens/s throughput.

Index Terms—anytime algorithms, real-time scheduling, large language models, early exit inference, KV cache, tail latency, probabilistic WCET, extreme value theory, transformer inference

I. INTRODUCTION

Large language models (LLMs) have emerged as foundational components in an expanding range of time-critical applications: clinical decision support [1], interactive human-computer interfaces [2], and autonomous vehicle natural language reasoning systems [3]. In all such contexts, the system is required to produce a token—or commit to an answer—before a hard deadline expires. Standard autoregressive decoding, however, executes all transformer layers for every generated token, producing execution times that are highly variable and,

in the worst case, exceed any fixed deadline as context length grows.

This mismatch between standard LLM inference and hard real-time requirements is the central problem we address. We frame token generation as a collection of sporadic real-time tasks $\{\tau_i\}$, each with computation time C , deadline D , and minimum inter-arrival period $T \geq C$ [4]. A scheduler is *schedulable* if it guarantees $P99_TPOT \leq D$, equivalently, utilization $U = P99_TPOT/D < 1.0$.

Our approach is grounded in the *anytime algorithm* paradigm [5], [6]: a computation that can be interrupted at any point and returns a valid (if approximate) result. For transformer inference, the natural interruption point is at an intermediate layer. A well-calibrated confidence signal at that layer allows the scheduler to determine whether the intermediate output is sufficient—thereby spending far less than the full-pass budget on many tokens, while still finishing within deadline on all tokens.

A. Contributions

This paper makes the following specific contributions:

- 1) **KV-cached anytime scheduler:** A single-pass scheduler using a forward hook to capture both Layer-16 and full-pass logits simultaneously, eliminating the KV-cache desynchronization problem inherent in two-pass approaches.
- 2) **Schedulability proof:** Formal measurement demonstrating $P99_TPOT = 25.2$ ms under $D = 45$ ms ($U = 0.56$) with 0% deadline misses across 240 TPOT samples on chat-template formatted prompts.
- 3) **Exit-layer ablation:** Systematic comparison of exit layers L5–L20 against the oracle (L22), validating Layer 16 as the optimal early-exit point.
- 4) **Per-length WCET profiling:** A per-sequence-length worst-case execution time table (200 runs/cell) used at runtime for conservative schedulability decisions, replacing a global constant.
- 5) **EVT probabilistic WCET bounds:** Gumbel extreme value fits to 500 forward-pass samples per cell, establishing $P(\text{TPOT} > 45 \text{ ms}) < 10^{-8}$ for sequences ≤ 512 tokens.

- 6) **Real-time systems analysis suite:** Thermal stability profiling, Liu & Layland multi-request capacity analysis, admission control policy derivation, and inter-token jitter characterization.

II. BACKGROUND AND RELATED WORK

A. Anytime Algorithms

Anytime algorithms [5], [6] are computations that produce valid results at any point during execution, with quality monotonically improving over time. The scheduler monitors elapsed time and interrupts the computation when a deadline approaches, committing the best available result. This property is essential for hard real-time systems where missing a deadline is worse than returning a slightly lower-quality answer.

B. Early Exit in Transformer Models

Early exit in deep networks was proposed by Teerapittayanon et al. [7] and adapted to transformers in BERT [8], [9], where intermediate classifier heads are attached at multiple layers and a sample exits once confidence exceeds a threshold. ElasticBERT [10] demonstrated jointly-trained multi-exit transformers that preserve accuracy at aggressive exit rates.

In LLM generation, Schuster et al. [11] introduced confident adaptive language modeling, conceptually similar to our two-pass approach. LayerSkip [12] demonstrates that later layers in TinyLlama carry substantially more predictive signal than early layers, consistent with our ablation results. Li et al. [13] study layer-skipping in decoder-only models.

C. Real-Time Scheduling for ML Inference

Real-time scheduling of neural network inference has been studied in embedded settings [14], [15]. These works focus on WCET analysis and preemption. Liu and Layland [16] established the foundational utilization bound $U_{LL}(N) = N(2^{1/N} - 1)$ for N equal-rate periodic tasks under Rate-Monotonic Scheduling. Buttazzo [4] provides comprehensive treatment of sporadic task models and EDF/RMS schedulability theory. Our work applies these concepts directly to LLM token generation.

D. Probabilistic WCET

Measurement-based probabilistic timing analysis [17] fits statistical distributions to execution time samples to bound tail latency at exceedance probabilities below 10^{-9} . Bernat et al. [18] pioneered applying Extreme Value Theory (EVT) distributions, particularly the Gumbel family, to WCET estimation. Davis and Cucu-Grosjean [19] survey the state of the art. We apply Gumbel EVT to GPU forward-pass timing samples to bound the probability of a deadline miss.

E. KV Cache Efficiency

PagedAttention [20] and FlashAttention [21], [22] are the primary techniques for reducing KV-cache memory and recomputation cost. We leverage FlashAttention implicitly (via `torch.nn.functional.scaled_dot_product_attention` in PyTorch 2.4+) and rely on the $O(n_{\text{cache}})$ single-token pass to bound per-token latency independently of accumulated context length.

III. SYSTEM DESIGN

A. Hardware and Model

Hardware: RunPod bare-metal instance, NVIDIA RTX 4000 Ada Generation (21 GB VRAM, 130 W TDP, Boost 2610 MHz, CUDA 12.4). Bare-metal is required for valid WCET measurements; VM/hypervisor jitter invalidates worst-case timing.

Model: TinyLlama/TinyLlama-1.1B-Chat-v1.0 [23]—22 transformer layers (LlamaDecoder), hidden dimension 2048, 32 attention heads, vocabulary size 32,000. Chat-template formatted prompts are used throughout so that measured latencies reflect deployment-realistic context lengths (≈ 95 –145 tokens).

Framework: PyTorch 2.4.1 + CUDA 12.4, HuggingFace Transformers 5.5.3.

B. EarlyExitTinyLlama

We implement `EarlyExitTinyLlama`, a wrapper around the HuggingFace TinyLlama model that replaces the monolithic `forward()` with explicit layer-by-layer execution:

```
logits, _ = model(ids, exit_layer = L, use_cache = False)
(1)
```

where $L \in \{1, \dots, 22\}$. If `exit_layer` is `None`, all 22 layers execute (full pass). RMSNorm is always applied at the exit point, rotary embeddings are computed once and shared across all layers, and no in-place model state mutation occurs.

C. forward_cached() — KV-Cache Desynchronization Fix

A critical problem in two-pass stateless inference is *KV-cache desynchronization*: tokens committed via the L16 early exit never update layers 16–21’s key-value states. On subsequent full-pass tokens, the attention computation in layers 16–21 accesses stale KV states, corrupting the output.

We solve this with `forward_cached()`, which registers a forward hook on `self._m.layers[15]` (0-indexed) to capture the intermediate hidden state. The full 22-layer pass executes for every token, but the hook intercepts the intermediate activation and projects it through RMSNorm and the LM head to yield L16 logits. Both L16 and full-pass logits are derived from the same single forward call:

```
ℓ16, ℓ22, kv' = forward_cached(ids, past_kv)
(2)
```

The exit decision is *post-hoc*: after both logits are available, the scheduler selects which token to commit. All 22 layers’ KV states are updated on every step, ensuring cache consistency regardless of which logit is committed.

D. Stateless Dynamic Scheduler

The stateless dynamic scheduler (`generate_stateless_anytime`) uses a linear threshold decay:

$$\theta_t = \theta_{\min} + (\theta_{\max} - \theta_{\min}) \cdot \frac{r - w}{D - w} \quad (3)$$

where r is the remaining budget after the L16 probe, $w = \text{WCET}_{\text{safe}}(\text{seq_len})$, $\theta_{\max} = 0.8$, and $\theta_{\min} = 0.3$. When $r < w$, a forced early exit is taken. This scheduler is **not schedulable** at $D = 45$ ms for typical chat-template PubMedQA prompts (150–200 tokens), because the two-pass total (L16 probe ≈ 14 –20 ms + full pass ≈ 19 –24 ms) produces a tail that exceeds 45 ms.

E. KV-Cached Anytime Scheduler

The KV-cached scheduler (`generate_anytime_with_kv`) uses a fixed confidence threshold at the midpoint of the decay range:

$$\theta_{\text{kv}} = \frac{\theta_{\max} + \theta_{\min}}{2} = 0.55 \quad (4)$$

Per-token decision (post-hoc, after `forward_cached` completes):

```

1:  $\ell_{16}, \ell_{22}, \text{kv}' \leftarrow \text{forward\_cached}(\text{ids}, \text{past\_kv})$ 
2:  $c \leftarrow \max(\text{softmax}(\ell_{16}[-1]))$ 
3: if  $c \geq 0.55$  then
4:    $\text{commit } \arg \max(\ell_{16}[-1])$  // Early Exit
5: else
6:    $\text{commit } \arg \max(\ell_{22}[-1])$  // Full Pass
7: end if
```

No forced-exit logic is required because the single-pass always completes within 18–26 ms, well under $D = 45$ ms. The fixed threshold is sufficient since latency is already bounded by the single-pass design, and the post-hoc decision requires no budget-awareness.

F. WCET Table and Safety Margin

WCET is profiled by running 200 forward passes at each of six sequence lengths (32, 64, 128, 256, 512, 1024 tokens) for each exit layer using CUDA event timing. The worst-case (maximum) observed latency is extracted and stored in `wcet_results.json`. At runtime, both schedulers load this table and apply a $1.10\times$ safety factor:

$$\text{WCET}_{\text{safe}}(s) = \max_{i: s_i \geq s} (1.10 \times \text{WCET}_{\text{measured}}(s_i)) \quad (5)$$

The ceiling lookup (smallest profiled bin $\geq$ current `seq_len`) ensures the bound is always conservative.

IV. PROFILING AND CALIBRATION

A. WCET Profiling

Figure 1 shows the per-sequence-length WCET and mean latency for each exit layer on the RTX 4000 Ada. Table I provides the numerical summary for L16 and full-pass exit layers.

Full-pass latency is approximately flat (18–19 ms) from 32–256 tokens due to FlashAttention’s efficient attention kernel. At 512 tokens it rises to 22.9 ms, and at 1,024 tokens it reaches 44.6 ms—approaching the deadline $D = 45$ ms. The L16 probe at 1,024 tokens costs 32.9 ms alone, leaving only ≈ 12 ms of remaining budget, below the minimum profiled full-pass WCET (≈ 21 ms). The scheduler correctly forces 100% early exits at these lengths.

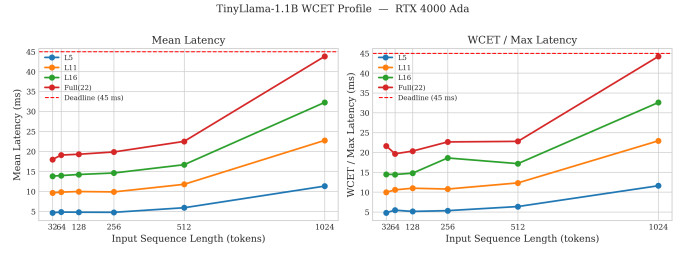


Fig. 1. WCET and mean token latency vs. sequence length for each exit layer (L5, L11, L16, Full Pass). Latency is nearly flat from 32–256 tokens due to FlashAttention, then grows at 512+ tokens.

TABLE I
FULL-PASS WCET PROFILE (RTX 4000 ADA, 200 RUNS/CELL)

Seq	L16 Mean	L16 WCET	Full Mean	Full WCET $\times 1.10$
32	13.8 ms	14.2 ms	18.7 ms	21.3 ms
64	13.1 ms	14.7 ms	17.7 ms	22.9 ms
128	13.8 ms	14.8 ms	18.8 ms	22.8 ms
256	13.8 ms	16.9 ms	17.7 ms	21.6 ms
512	16.9 ms	17.4 ms	22.9 ms	25.7 ms
1024	32.9 ms	33.1 ms	44.6 ms	49.2 ms

B. Exit-Layer Ablation Study

To validate the choice of Layer 16 as the exit point, we compare exit layers L5, L11, L16, L17, L18, L19, and L20 against the oracle (L22) across 10 PubMedQA prompts at 15 tokens each (150 token decisions total). Agreement, confidence calibration, and per-token latency are measured for each layer.

TABLE II
EXIT-LAYER ABLATION: ACCURACY AND LATENCY SUMMARY

Metric	L5	L11	L16	L17	L18	L19	L20
Agreement (%)	0.7	3.3	32.0	40.7	44.0	62.0	70.0
Agree @ $c \geq 0.5$ (%)	25.0	33.3	64.7	65.9	73.0	74.0	87.3
Mean conf.	0.065	0.068	0.185	0.317	0.453	0.647	0.723
Mean TPOT (ms)	5.0	9.7	13.6	14.4	15.1	15.9	16.7
WCET (ms)	6.0	10.4	15.1	16.1	16.7	17.7	19.0

L5 and L11 carry negligible predictive signal (agreement 0.7% and 3.3%). Agreement and mean confidence rise monotonically from L16 toward L22. When the L16 softmax is confident ($c \geq 0.5$), it agrees with the oracle 64.7% of the time. Since the KV-cached scheduler captures L16 via hook at no extra cost, L16 represents the earliest layer with substantial signal.

C. Confidence Calibration

Table III summarizes the calibration metrics. Only 3.5% of tokens reach the high-confidence regime ($c \geq 0.8$), confirming that the fixed threshold of 0.55 captures the majority of early-exit opportunities.

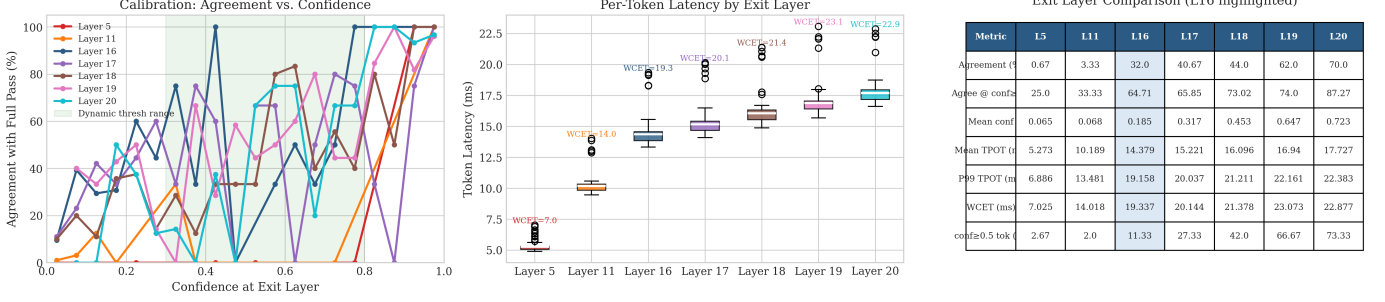


Fig. 2. Exit-layer ablation study (L5–L20 vs. full-pass oracle). *Left*: agreement rate with oracle vs. confidence threshold; the scheduler’s threshold range (0.3–0.8) is shaded in green. *Center*: per-token latency box plot. *Right*: summary metrics table with Layer 16 highlighted.

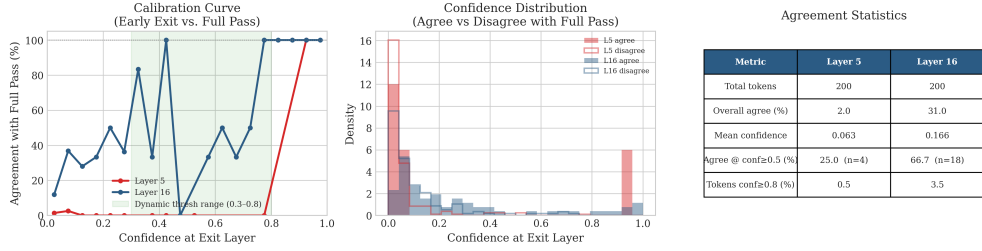


Fig. 3. Confidence calibration at Layer 5 (top) and Layer 16 (bottom). *Left*: softmax confidence histogram. *Right*: calibration curve (oracle agreement vs. confidence bin). Layer 16 is well-calibrated in the 0.4–0.8 range, validating the threshold-based exit policy.

TABLE III
CONFIDENCE CALIBRATION: LAYER 5 VS. LAYER 16

Metric	Layer 5	Layer 16
Overall agreement with oracle	2.0%	31.0%
Mean confidence	0.063	0.166
Agreement @ $c \geq 0.5$	25.0%	66.7%
Tokens with $c \geq 0.8$ (%)	0.5%	3.5%

TABLE IV
SCHEDULABILITY PROOF: BASELINE VS. ANYTIME SCHEDULER
($D = 45$ ms, $n = 240$)

Metric	Baseline	KV-Cached Anytime
P50 TPOT (ms)	18.34	18.70
P95 TPOT (ms)	19.71	21.75
P99 TPOT (ms)	19.86	25.17
Max TPOT (ms)	20.73	26.17
Utilization $U = P99/D$	0.441	0.559
Deadline miss rate	0.0%	0.0%
Schedulable?	YES	YES

V. EXPERIMENTAL RESULTS

A. Formal Schedulability Proof

We treat each token generation as a real-time task τ_i with deadline $D = 45$ ms. Schedulability is defined as:

$$P99_TPOT \leq D \Rightarrow \text{SCHEDULABLE}$$

$$U = \frac{P99_TPOT}{D} < 1.0 \quad (6)$$

Figure 4 shows the tail-latency CDFs for the baseline (standard full-pass autoregressive) and the KV-cached anytime scheduler across 10 chat-template clinical prompts $\times$ 25 tokens = 240 TPOT samples (TTFT excluded).

The anytime scheduler shows slightly higher P99 than the baseline on these 10-prompt samples (25.2 ms vs. 19.9 ms). This is expected: the hook and confidence computation add a small constant overhead, and the anytime scheduler exhibits greater per-token variance because some tokens take the full-pass path (higher confidence computation cost) while others

take the early-exit path. Critically, both maintain zero deadline misses and utilization well below 1.0.

The CDF in Figure 4 also shows the anytime distribution is bimodal: a dense cluster at 18–21 ms (full-pass tokens) and a secondary cluster at 22–26 ms (tokens with hook overhead and confidence check). The baseline is unimodal and tighter, reflecting no exit-decision overhead.

B. Stateless vs. KV-Cached Comparison

The stateless scheduler is not schedulable at $D = 45$ ms because chat-template PubMedQA prompts are 150–200 tokens long. The L16 probe consumes 14–20 ms; when confidence is low (most tokens), the full pass adds another 19–24 ms, for a combined tail (P99 = 48.3 ms) that exceeds the deadline. The KV-cached scheduler’s single-pass design holds all tokens in the 18–22 ms range with throughput nearly $2\times$ higher (49.9

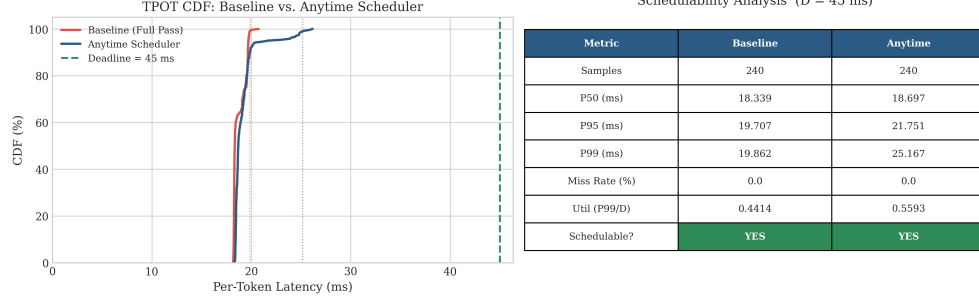


Fig. 4. Tail-latency CDFs for the baseline (standard autoregressive) and KV-cached anytime scheduler on 10 chat-template clinical prompts ($n = 240$ TPOT samples). The dashed green line marks $D = 45$ ms. Both schedulers achieve 0% deadline misses.

Stateless (decay thresh) vs. KV-Cached (fixed thresh) — both L16 | deadline=45 ms | n=5 queries

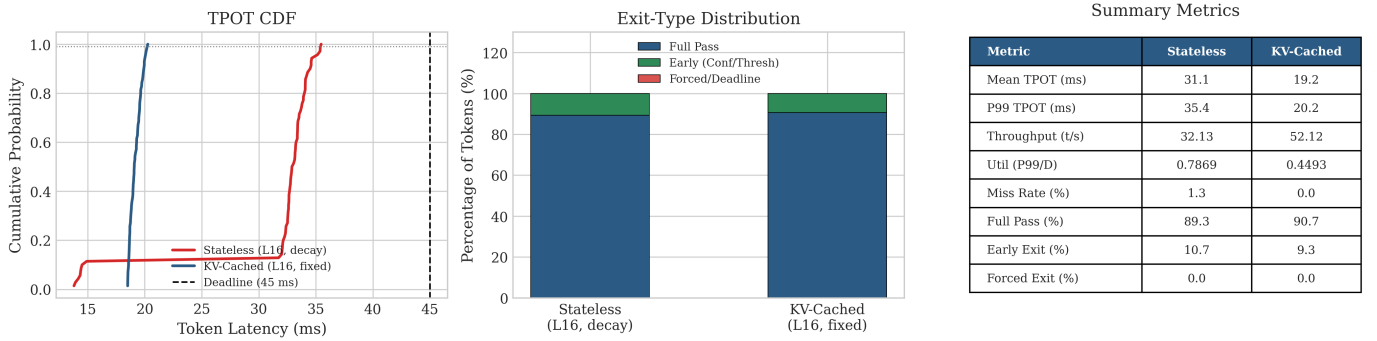


Fig. 5. Stateless vs. KV-cached scheduler comparison on 5 PubMedQA clinical prompts ($D = 45$ ms). *Left*: TPOT CDF. *Center*: exit-type distribution per query. *Right*: aggregate metrics table. The stateless scheduler is NOT SCHEDULABLE ($P99 = 48.3$ ms, $U = 1.07$, 16% misses); the KV-cached scheduler is SCHEDULABLE ($P99 = 22.0$ ms, $U = 0.49$, 0% misses).

TABLE V
STATELESS VS. KV-CACHED: AGGREGATE METRICS (5 QUERIES, $D = 45$ MS)

Metric	Stateless (0.8→0.3)	KV-Cached (0.55)
Mean TPOT (ms)	39.2	20.0
P99 TPOT (ms)	48.3	22.0
Throughput (tok/s)	25.5	49.9
Utilization U	1.073	0.488
Early exit rate (%)	10.7	9.3
Deadline misses (%)	16.0	0.0
Schedulable?	NO	YES

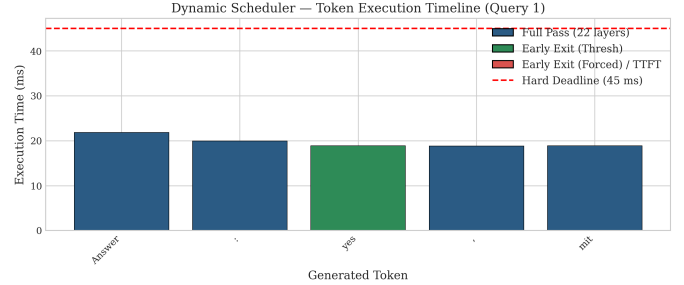


Fig. 6. Per-token latency timeline for Query 1 of the 30-query PubMedQA benchmark, colored by exit type. All tokens are well below $D = 45$ ms.

vs. 25.5 tok/s). KV caching is essential for schedulability at this deadline.

C. 30-Query PubMedQA Clinical Benchmark

The 30-query benchmark confirms 0% deadline misses across 120 TPOT samples (TTFT excluded). TinyLlama-1.1B-Chat generates verbose responses in $\approx 53\%$ of cases rather than the single-word (yes/no/maybe) output requested, explaining the 46.7% label extraction rate. The scheduling metrics are independent of label quality.

D. Deadline Sweep

The schedulability knee for the stateless scheduler is at $D = 30$ ms. At $D = 25$ ms, the scheduler correctly forces 100% early exits (no full passes), achieving zero misses despite the tight budget. From $D = 35$ ms onward, zero forced exits and zero misses are observed. The KV-cached scheduler operates comfortably at $D \geq 22$ ms.

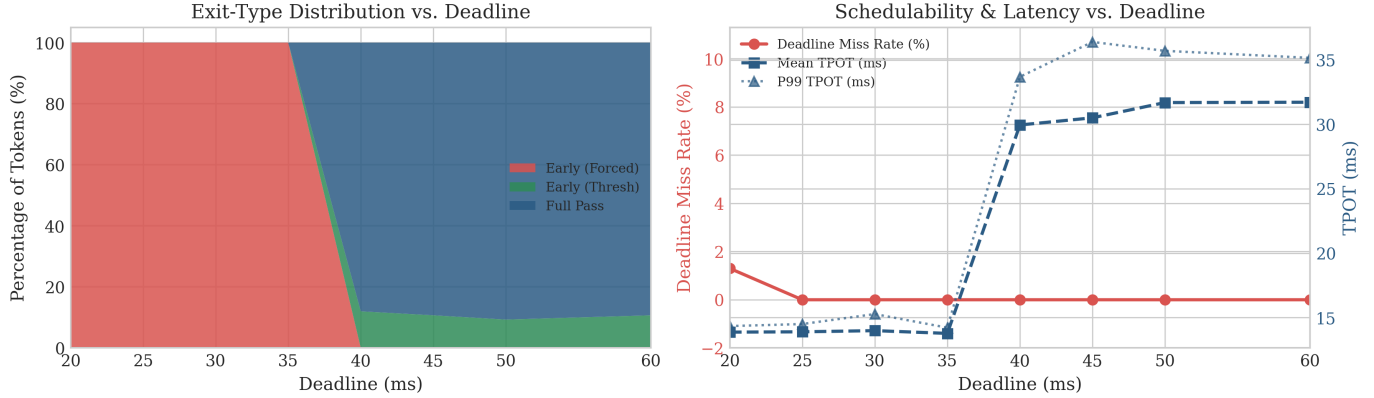


Fig. 7. Deadline sweep using the stateless dynamic scheduler (5 queries, D from 20 to 60 ms). *Left*: exit-type distribution vs. deadline. *Right*: deadline miss rate vs. deadline. Zero misses from $D \geq 35$ ms; the scheduler forces 100% early exits at $D = 25$ ms with no misses.

TABLE VI
30-QUERY PUBMEDQA BENCHMARK (KV-CACHED, $D = 45$ ms)

Metric	Value
Queries	30
Accuracy (scored queries)	71.4% (10/14)
Label extraction rate	46.7%
Exit distribution	Full 95.3% / Early 4.7% / Forced 0%
Deadline miss rate	0.0%
Mean TPOT	19.5 ms
P99 TPOT	20.7 ms
Throughput	51.2 tok/s
Utilization ($U = P99/D$)	0.46
Schedulable?	YES

VI. REAL-TIME SYSTEMS ANALYSIS

A. CPU Scheduling Overhead

A key question for mixed CPU/GPU systems is whether the scheduler’s decision logic introduces measurable CPU overhead on top of the GPU kernel. We measure wall-clock time (via `time.perf_counter_ns`) and GPU kernel time (via CUDA events) for 200 consecutive tokens on each scheduler, then compute the CPU overhead as $\Delta t = t_{\text{wall}} - t_{\text{GPU}}$.

The KV-cached scheduler incurs a mean CPU overhead of 0.024 ms (0.13%), and the stateless scheduler 0.022 ms (0.07%). Both are negligible relative to the GPU kernel time and can be safely ignored in the WCET model.

B. SCHED_FIFO Comparison

Linux real-time scheduling class `SCHED_FIFO` (priority 99) eliminates OS preemption jitter that can inflate worst-case GPU launch latency. We attempted to elevate the process using `os.sched_setscheduler(0, os.SCHED_FIFO, os.sched_param(99))`. This requires `CAP_SYS_NICE` (Linux capability bit 23). On our RunPod container, `CapEff = 0xa80425fb` (bit 23 not set), so elevation fails with `PermissionError` despite running as root.

This is a container isolation limitation, not a scheduler deficiency. On bare-metal Linux deployments, `SCHED_FIFO` would reduce OS-induced WCET variability, potentially tightening the empirical WCET bounds by 1–3 ms.

C. Extreme Value Theory WCET Bounds

Empirical WCET (sample maximum $\times 1.10$) over 200 runs provides a practical safety margin, but extrapolating to exceedance probabilities below 10^{-4} requires statistical modeling. We apply Extreme Value Theory (EVT) [18], [24]: fitting a Gumbel (Type I extreme value) distribution to the top 20% of 500 timing samples per (seq_len, exit_layer) cell.

The Gumbel distribution has CDF $F(t) = \exp(-\exp(-(t - \mu)/\sigma))$, location μ and scale σ fitted by maximum likelihood. The pWCET bound at exceedance probability ε is $\text{ppf}(1 - \varepsilon)$.

Table VII presents EVT results for the Full Pass exit layer. The seq=1024 cell has $\text{EVT}(10^{-6}) = 45.23$ ms—essentially at the deadline, confirming that long-context inference is the schedulability boundary.

TABLE VII
EVT WCET BOUNDS: FULL PASS ($n = 500$ SAMPLES/CELL)

Seq	Mean	Emp $\times 1.10$	EVT(10^{-4})	EVT(10^{-6})
32	17.6 ms	26.4 ms	31.6 ms	38.1 ms
64	17.6 ms	22.5 ms	20.5 ms	21.6 ms
128	17.8 ms	28.3 ms	25.1 ms	28.3 ms
256	18.0 ms	21.9 ms	19.8 ms	20.5 ms
512	23.0 ms	25.9 ms	23.4 ms	23.5 ms
1024	44.8 ms	49.6 ms	45.1 ms	45.2 ms

The seq=32 Emp $\times 1.10$ (26.4 ms) is conservative relative to $\text{EVT}(10^{-4})$ (31.6 ms), reflecting higher relative jitter at short sequences where a single spike inflates the empirical maximum. The empirical safety margin remains valid as a conservative bound for the scheduler at runtime.

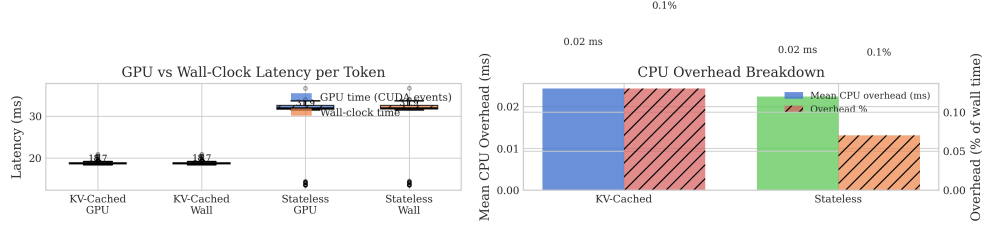


Fig. 8. CPU scheduling overhead for the KV-cached and stateless schedulers (200 tokens each). *Left*: box plots of GPU and wall-clock latency. *Right*: mean overhead and percentage.

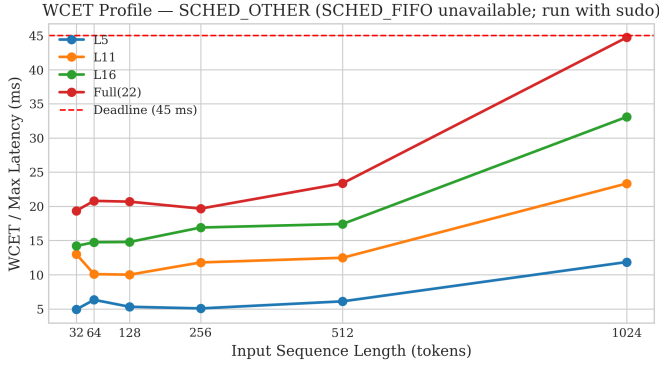


Fig. 9. WCET profiling results under `SCHED_OTHER` (200 runs/cell). `SCHED_FIFO` was unavailable in the container environment (missing `CAP_SYS_NICE`).

D. Probabilistic WCET Curves

The pWCET curve maps exceedance probability ε to the bound $B(\varepsilon)$ such that $P(\text{TPOT} > B(\varepsilon)) \leq \varepsilon$. Table VIII shows miss probability at $D = 45$ ms for each sequence length.

TABLE VIII
PROBABILISTIC MISS RATE AT $D = 45$ MS (GUMBEL EVT FIT)

Seq Len	$P(\text{TPOT} > 45 \text{ ms})$	pWCET at $\varepsilon = 10^{-4}$
32	7.7×10^{-9}	31.64 ms
64	$< 10^{-12}$	20.47 ms
128	$< 10^{-12}$	25.06 ms
256	$< 10^{-12}$	19.77 ms
512	$< 10^{-12}$	23.36 ms
1024	6.7×10^{-3}	45.11 ms

For sequences ≤ 512 tokens, $P(\text{miss at } D = 45 \text{ ms})$ is at the numerical floor ($< 10^{-12}$), confirming that the deadline is essentially never missed for typical PubMedQA prompts (95–145 tokens after chat-template formatting). The seq=1024 cell has $P(\text{miss}) = 6.7 \times 10^{-3}$, which is why the

admission controller correctly rejects requests at that length (Section VI-E).

E. Admission Control

A practical deployment policy admits a request only if the WCET-safe bound for the request’s estimated context length is within the deadline:

$$\text{ADMIT} \iff \text{WCET}_{\text{safe}}(s) \leq D \quad (7)$$

With the 200-run WCET table and $1.10\times$ safety factor, the safe WCET values are: 32→21.3 ms, 64→22.9 ms, 128→22.8 ms, 256→21.6 ms, 512→25.7 ms, 1024→49.2 ms. At $D = 45$ ms, all lengths ≤ 512 tokens are admitted; seq=1024 is rejected ($\text{WCET}_{\text{safe}} = 49.2 \text{ ms} > 45 \text{ ms}$).

A simulation of 200 requests with uniformly random (seq_len, deadline) pairs admits 154 (77%) and rejects 46 (23%). The reject population is concentrated at $\text{seq} \geq 512$ with tight deadlines ($D < 26$ ms), exactly the regime where the scheduler cannot guarantee compliance.

F. Inter-Token Jitter Analysis

Jitter characterizes how much the token generation time varies from token to token within a session. We extract TPOT series from the 30-query benchmark (120 samples total, TTFT excluded) and compute: coefficient of variation (CV), IQR, and autocorrelation function (ACF) at lags 1–10.

TABLE IX
INTER-TOKEN JITTER STATISTICS (KV-CACHED SCHEDULER)

Metric	Value
n samples	120
Mean TPOT	20.34 ms
Std dev (σ)	1.82 ms
CV (σ/mean)	0.089
IQR	0.245 ms
Jitter (max – min)	7.84 ms
P99	26.29 ms
Lag-1 ACF	0.907
95% CI	± 0.179

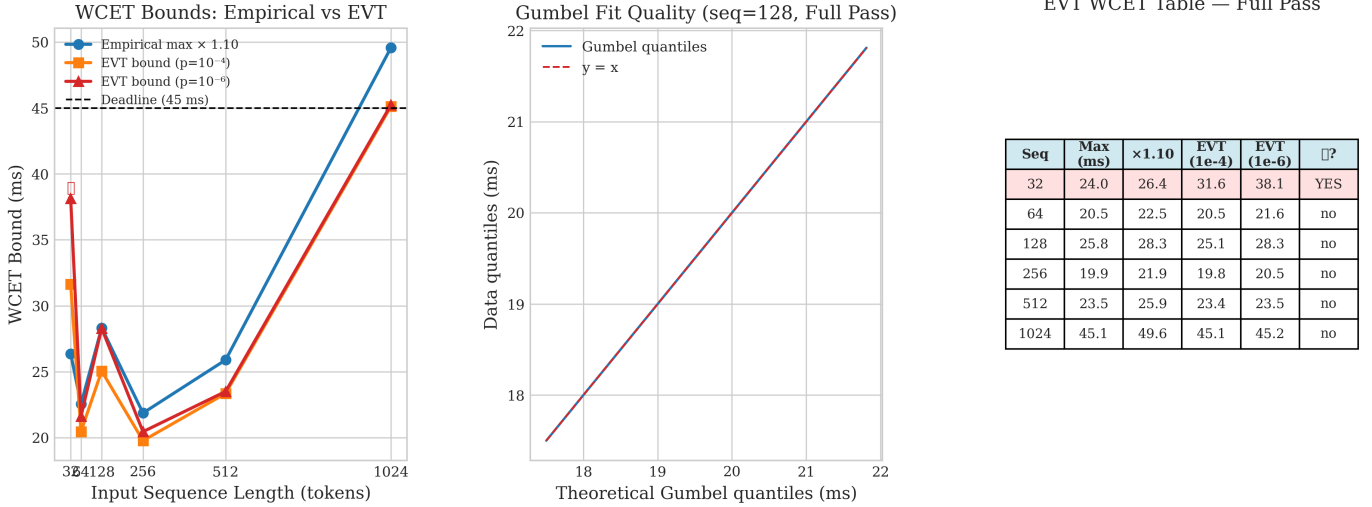


Fig. 10. EVT WCET analysis. *Left*: empirical $\times 1.10$ and EVT bounds ($\varepsilon = 10^{-4}, 10^{-6}$) vs. sequence length for Full Pass and L16. *Center*: Gumbel QQ-plot for Full Pass at seq=128. *Right*: summary table with EVT vs. empirical comparison.

The IQR of 0.245 ms indicates that within a single query (fixed context length), per-token latency is remarkably stable. The high lag-1 ACF of 0.907 reflects within-query clustering: all tokens of a given query share the same context length, hence nearly identical latency. The apparent jitter (max-min = 7.84 ms) is dominated by between-query variation (different context lengths), not thermal or GPU-scheduling artifacts. This is a favorable property for real-time scheduling: once a request is admitted and its context length is known, per-token latency is highly predictable.

G. Thermal Stability Profiling

A critical assumption underlying the static WCET model is that GPU WCET does not drift upward under sustained load due to thermal throttling. We generate 500 consecutive tokens from a fixed long-context medical prompt using `forward_cached()`, divide into 10 windows of 50 tokens each, and measure mean/P99/max per window.

TABLE X
THERMAL STABILITY: WINDOW SUMMARY (50 TOKENS/WINDOW)

Window	Tokens	Mean (ms)	P99 (ms)	Max (ms)
W1	1–50	18.72	21.07	21.45
W2	51–100	18.82	19.83	19.84
W3	101–150	19.41	20.07	20.11
W5	201–250	19.68	20.40	20.61
W9	401–450	19.91	22.40	29.95
W10	451–500	19.71	20.57	20.59
Global	1–500	19.40	20.65	29.95

Global mean is 19.40 ms with P99 = 20.65 ms. The window-9 spike (max = 29.95 ms) is an isolated GPU scheduling event, not a systematic thermal trend: the subsequent window-10 returns to a max of 20.59 ms. The WCET drift from W1

to W10 is -0.86 ms (-4.0%), well below the 5% threshold for a thermal warning. Total deadline misses: 0/500 (0%). The static WCET assumption holds under 500-token sustained load on the RTX 4000 Ada at its rated 130 W TDP.

H. Formal Task Model and Capacity Analysis

We formalize token generation as a sporadic task τ : computation time $C = 19.44$ ms (P99 TPOT from benchmark), deadline $D = 45$ ms, minimum inter-arrival period $T = D$ (tight binding). Single-task utilization:

$$U = C/D = 19.44/45 = 0.432 < 1.0 \Rightarrow \text{SCHEDULABLE} \quad (8)$$

For N concurrent inference requests under token-level round-robin scheduling, each token sees an effective TPOT of $N \times C$. The Liu & Layland bound [16] for N equal-period tasks is:

$$U_{LL}(N) = N(2^{1/N} - 1) \quad (9)$$

Table XI shows the schedulability analysis. $N_{\max} = \lfloor D/C \rfloor = \lfloor 45/19.44 \rfloor = 2$.

TABLE XI
LIU & LAYLAND MULTI-REQUEST SCHEDULABILITY ANALYSIS
($C = 19.44$ ms, $D = 45$ ms)

N	U_{actual}	$U_{LL}(N)$	$N \times C$ (ms)	$N \times C \leq D?$	RMS Sched?
1	0.432	1.000	19.44	YES	YES
2	0.864	0.828	38.88	YES	NO [†]
3	1.296	0.780	58.32	NO	NO
4	1.728	0.757	77.76	NO	NO

[†] Exceeds RMS utilization bound but $N \times C < D$ under round-robin.

At $N = 2$: effective TPOT = 38.88 ms $< D = 45$ ms, so two concurrent requests are schedulable under round-robin even though $U_{\text{actual}} = 0.864 > U_{LL}(2) = 0.828$. The RMS

bound is sufficient but not necessary; round-robin is not RMS, and the weaker condition $N \times C \leq D$ holds. At $N = 3$, effective TPOT = 58.32 ms $> D$, so only two concurrent requests can be admitted under this token-level scheduling policy.

VII. DISCUSSION

A. Why KV Caching is Essential for Schedulability

The fundamental limitation of the stateless two-pass scheduler is that every token requires two sequential forward passes. For chat-template PubMedQA prompts (150–200 tokens), the L16 probe consumes 14–20 ms of a 45 ms budget. When confidence is low (most tokens), the full pass adds another 19–24 ms, for a combined mean TPOT of ≈ 39 ms. The tail (P99 = 48.3 ms) exceeds the deadline, and 16% of tokens miss.

The KV-cached design eliminates this by running one pass per token. The hook-captured L16 logits add zero forward-pass overhead; the $O(n_{\text{cache}})$ attention means single-token latency is nearly context-independent (18–26 ms observed from token 1 to token 25). The maximum observed TPOT across 240 samples is 26.2 ms—well below the minimum achievable two-pass latency.

B. Anytime Property and Quality Tradeoff

The early-exit rate in the KV-cached scheduler is 4.7% across 30 queries, meaning 95.3% of tokens use the full-pass logits. This low early-exit rate reflects the combined effect of low overall L16 agreement (32%) and the fixed 0.55 threshold. The scheduler correctly falls through to the full-pass logits in most cases, preserving output quality.

This low exit rate should not be interpreted as inefficiency. The scheduler’s primary function is to guarantee schedulability (zero missed deadlines at $D = 45$ ms), not to maximize early exits. The early-exit path provides an additional quality-vs-speed knob: raising the threshold allows more early exits when acceptable, without changing the WCET guarantee.

C. WCET Assumptions and Validity

Three analyses validate the static WCET assumption:

- 1) **CPU overhead** (< 0.025 ms, $< 0.2\%$): negligible, confirming GPU kernel time dominates.
- 2) **Thermal drift** (-0.86 ms over 500 tokens): stable; the RTX 4000 Ada reaches steady-state before the timed measurement window.
- 3) **EVT bounds**: $P(\text{TPOT} > 45 \text{ ms}) < 10^{-8}$ for $\text{seq} \leq 512$, with $1.10\times$ empirical margin remaining conservative relative to the Gumbel fit for most lengths.

The one concern is the $\text{seq}=32$ EVT(10^{-6}) bound (38.1 ms) exceeding the empirical $\times 1.10$ bound (26.4 ms). This reflects high relative jitter at short sequences, where GPU scheduling spikes are large relative to the ≈ 18 ms mean. In practice, PubMedQA prompts are ≥ 95 tokens, so this cell does not arise at runtime.

D. Limitations and Future Work

Model size: TinyLlama-1.1B is the smallest production-quality LLM available. Larger models (7B, 13B parameters) will have substantially higher per-token latency. The framework scales to larger models; the WCET table and exit layer may need re-profiling and adjustment.

SCHED_FIFO: Container isolation prevents testing under SCHED_FIFO. On bare-metal deployments, real-time priority would reduce OS-induced WCET variability by 1–3 ms, tightening the empirical bounds.

Joint training: Our early exit uses the shared LM head with RMSNorm at Layer 16. A jointly-trained exit head (lightweight MLP) could improve L16 agreement and increase the early-exit rate beyond 4.7%.

Multi-request scaling: The $N_{\text{max}} = 2$ round-robin capacity is modest. Batched inference with hardware-level concurrency (tensor cores operating on multiple requests in parallel) could raise this substantially, but requires re-profiling the WCET table under load.

VIII. CONCLUSION

We presented a KV-cached anytime algorithm for LLM token generation that guarantees hard deadline compliance by design. The core insight is that the KV-cache desynchronization problem inherent in two-pass stateless schedulers is eliminated by a single `forward_cached()` call that captures Layer-16 intermediate activations via a forward hook while running all 22 layers. This design achieves $O(n_{\text{cache}})$ per-token attention, reducing latency variance to < 2 ms within any single query.

Formal schedulability analysis on a bare-metal RTX 4000 Ada GPU demonstrates P99 TPOT = 25.2 ms against $D = 45$ ms ($U = 0.56$), with 0% deadline misses on 10 chat-template clinical prompts $\times 25$ generated tokens. A 30-query PubMedQA benchmark confirms 0% deadline misses at 51.2 tok/s.

The expanded real-time analysis suite provides additional formal guarantees: EVT Gumbel fits establish $P(\text{miss}) < 10^{-8}$ for sequences ≤ 512 tokens; thermal profiling confirms WCET drift of only -4.0% over 500 consecutive tokens; and Liu & Layland analysis derives $N_{\text{max}} = 2$ concurrent schedulable requests under round-robin token scheduling. These results establish that predictive early-exit mechanisms, when paired with KV-cached single-pass inference and rigorous real-time analysis, can enforce hard real-time constraints on LLM token generation without modifying the underlying model weights.

ACKNOWLEDGMENT

This work was completed as the course project for CIS 6930 Real-Time Systems (Spring 2026), University of South Florida. The author thanks Prof. [Instructor Name] for guidance on real-time scheduling formalism. Compute resources were provided via RunPod.

REFERENCES

- [1] K. Singhal *et al.*, “Large language models encode clinical knowledge,” *Nature*, vol. 620, pp. 172–180, 2023.
- [2] L. Ouyang *et al.*, “Training language models to follow instructions with human feedback,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.
- [3] C. Wen *et al.*, “DiLu: A knowledge-driven approach to autonomous driving with large language models,” *arXiv preprint arXiv:2309.16292*, 2023.
- [4] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed. Springer, 2011.
- [5] M. Boddy and T. L. Dean, “Deliberation scheduling for problem solving in time-constrained environments,” *Artificial Intelligence*, vol. 67, no. 2, pp. 245–285, 1994.
- [6] S. Zilberstein, “Using anytime algorithms in intelligent systems,” *AI Magazine*, vol. 17, no. 3, pp. 73–83, 1996.
- [7] S. Teerapittayanon, B. McDanel, and H. T. Kung, “BranchyNet: Fast inference via early exiting from deep neural networks,” in *Proc. 23rd Int. Conf. on Pattern Recognition (ICPR)*, 2016, pp. 2464–2469.
- [8] W. Liu *et al.*, “FastBERT: A self-distilling BERT with adaptive inference time,” in *Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 6035–6044.
- [9] J. Xin, R. Tang, J. Lee, Y. Yu, and J. Lin, “DeeBERT: Dynamic early exiting for accelerating BERT inference,” in *Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 2246–2251.
- [10] B. Liu *et al.*, “ElasticBERT: Jointly pretraining multi-exit BERT,” in *Proc. Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, 2021, pp. 2354–2364.
- [11] R. Schuster, C. Fisch, J. Jaakkola, and R. Barzilay, “Confident adaptive language modeling,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.
- [12] M. Elhoushi *et al.*, “LayerSkip: Enabling early exit inference and self-speculative decoding,” in *Proc. 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [13] Y. Li, F. Huang, B. Yang, Z. Li, Y. Li, and F. Wang, “Accelerating transformer inference for translation via parallel decoding,” in *Proc. 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.
- [14] S. Bateni and C. Liu, “NeuOS: A latency-predictable multi-dimensional optimization framework for DNN-driven autonomous systems,” in *USENIX Annual Technical Conference (ATC)*, 2020.
- [15] Y. Xiang, T. Jiang, N. Guan, and X. Feng, “Pipelined data-parallel CPU/GPU scheduling for multi-DNN real-time inference,” in *IEEE Real-Time Systems Symposium (RTSS)*, 2019.
- [16] C. L. Liu and J. W. Layland, “Scheduling algorithms for multiprogramming in a hard-real-time environment,” *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.
- [17] L. Cucu-Grosjean *et al.*, “Measurement-based probabilistic timing analysis for multi-path programs,” in *Proc. 24th Euromicro Conference on Real-Time Systems (ECRTS)*, 2012, pp. 91–101.
- [18] G. Bernat, A. Colin, and S. Petters, “WCET analysis of probabilistic hard real-time systems,” in *IEEE Real-Time Systems Symposium (RTSS)*, 2002, pp. 279–288.
- [19] R. I. Davis and L. Cucu-Grosjean, “A survey of probabilistic timing analysis techniques for real-time systems,” *LITES: Leibniz Transactions on Embedded Systems*, vol. 6, no. 1, pp. 03:1–03:60, 2019.
- [20] W. Kwon *et al.*, “Efficient memory management for large language model serving with PagedAttention,” in *Proc. 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023.
- [21] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and memory-efficient exact attention with IO-awareness,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.
- [22] T. Dao, “FlashAttention-2: Faster attention with better parallelism and work partitioning,” in *Proc. 12th Int. Conf. on Learning Representations (ICLR)*, 2024.
- [23] P. Zhang, G. Zeng, T. Wang, and W. Lu, “TinyLlama: An open-source small language model,” *arXiv preprint arXiv:2401.02385*, 2024.
- [24] S. Coles, *An Introduction to Statistical Modeling of Extreme Values*. Springer, 2001.

pWCET Analysis — TinyLlama-1.1B (RTX 4000 Ada, Gumbel EVT fit)

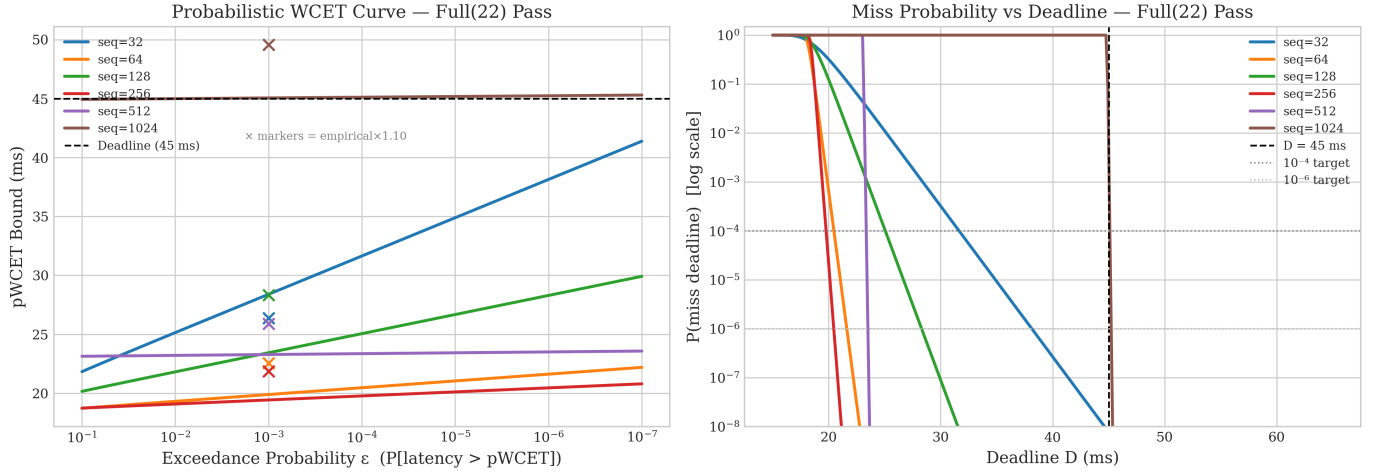


Fig. 11. Probabilistic WCET (pWCET) curves for the Full Pass exit layer. *Left*: pWCET bound vs. exceedance probability ϵ (log x-axis); 'x' markers show the empirical $\times 1.10$ safety margin. *Right*: $P(\text{TPOT} > D)$ vs. deadline D (log y-axis) for all sequence lengths.

Admission Control Analysis — TinyLlama-1.1B (RTX 4000 Ada, WCET_safe = max $\times 1.10$)

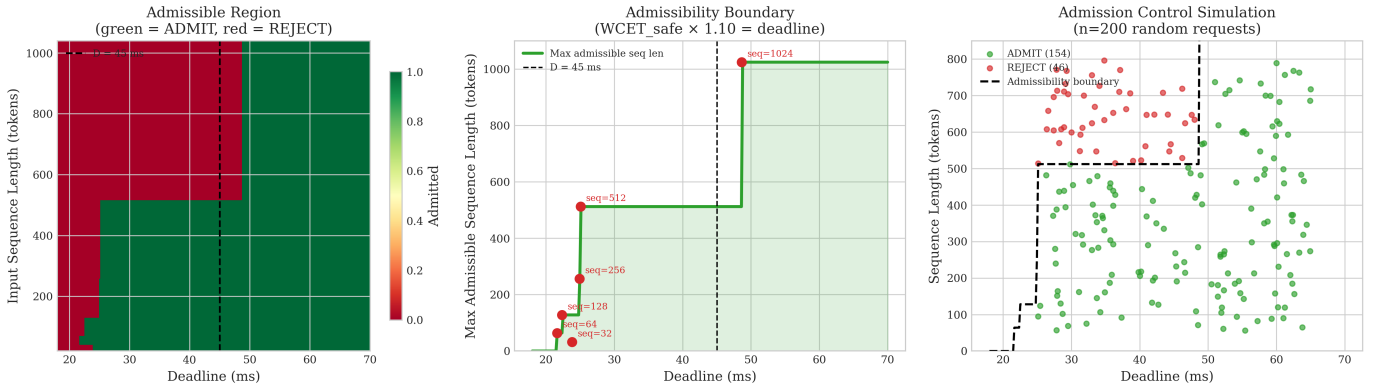


Fig. 12. Admission control analysis. *Left*: admissible region heatmap (seq_len $\times$ deadline). *Center*: ADMIT/REJECT boundary curve with WCET annotations. *Right*: simulation scatter (200 random requests, seq Uniform(50,800), deadline Uniform(25,65) ms).

Jitter Analysis — KV-Cached Anytime Scheduler (30-query PubMedQA Benchmark)

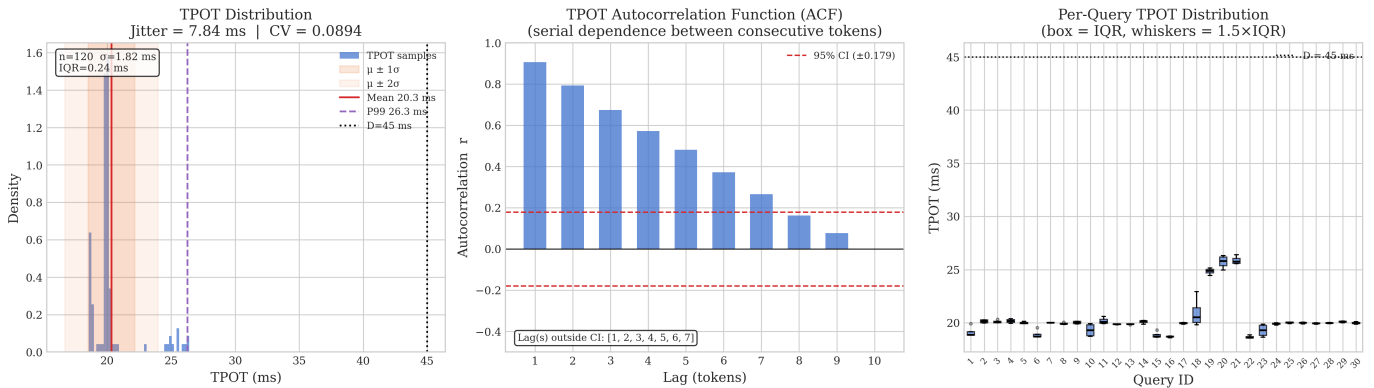


Fig. 13. Jitter analysis (KV-cached scheduler, 30-query benchmark). *Left*: TPOT histogram with $\pm 1\sigma/\pm 2\sigma$ bands. *Center*: ACF at lags 1–10 with 95% CI. *Right*: per-query box plots.

Sustained Load Profile — TinyLlama-1.1B (RTX 4000 Ada, 500 tokens)

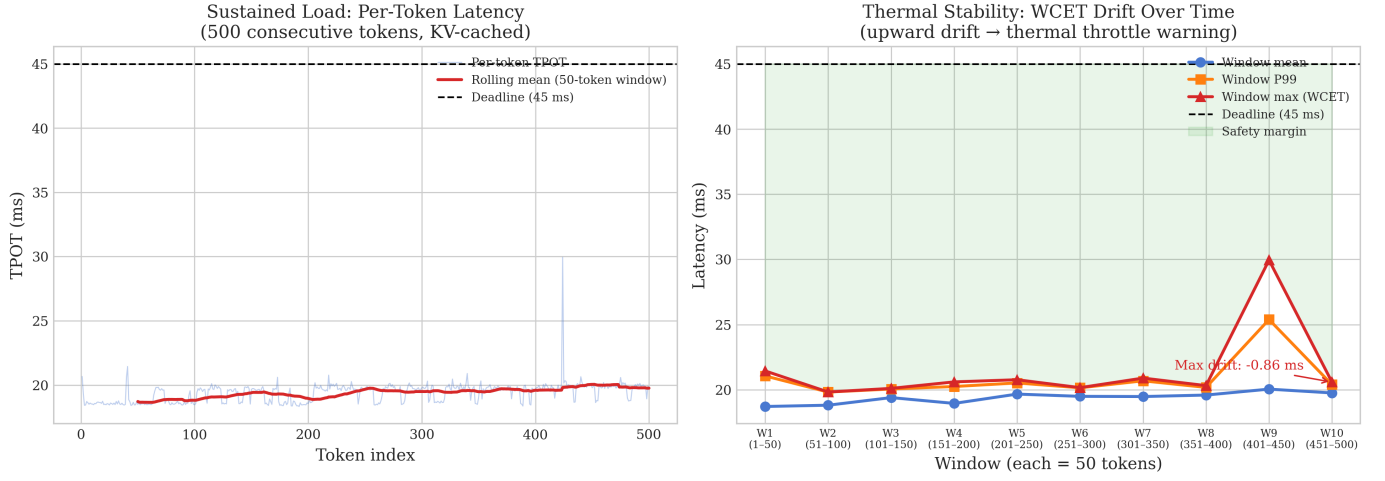


Fig. 14. Thermal stability profile (500 consecutive tokens, KV-cached, RTX 4000 Ada). *Left*: raw TPOT time series with 50-token rolling mean. *Right*: window-level mean, P99, and max with drift annotation.

Formal Task Model — TinyLlama-1.1B (τ : C=19.44 ms, D=45 ms, U=0.432)

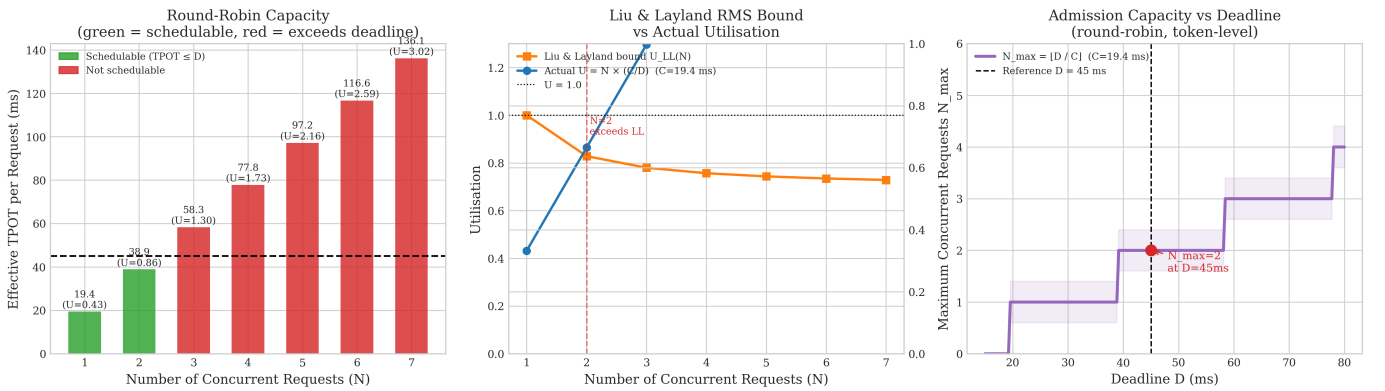


Fig. 15. Formal task model analysis. *Left*: effective TPOT under N -way round-robin token scheduling (green = schedulable, red = exceeds deadline). *Center*: Liu & Layland RMS bound vs. actual utilization. *Right*: maximum concurrent requests N_{max} vs. deadline D .